

```
"""
```

Project 3: The Random Password Generator

Goal:

Ask the user for a password length, then generate a random, complex password built from letters and numbers.

Key Skill:

Importing modules & string manipulation.

A note on module choice:

The brief's own one-line summary names Python's ``random`` module. But the brief's deeper technical content directly contradicts that: it explains ``random`` is a deterministic generator that can be predicted if its seed is known, and explicitly states that `secrets.choice()` must replace `random.choice()` for anything security-related. That's also the real-world standard Python's own documentation recommends, so this script uses ``secrets`` instead of ``random``.

```
"""
```

```
import string
```

```
import secrets
```

```
MINIMUM_RECOMMENDED_LENGTH = 12
```

```
def get_password_length():
```

```
    """
```

INPUT + validation: keep asking until we get a usable whole number. Bad input here should never crash the program - it should just ask again.

```
"""
```

```
while True:
```

```
    raw_input_text = input("How many characters should the password be? ").strip()
```

```
    try:
```

```
        length = int(raw_input_text)
```

```
    except ValueError:
```

```
        print("Please enter a whole number (e.g., 8).\n")
```

```
        continue
```

```
    if length <= 0:
```

```
        print("Length must be a positive number.\n")
```

```
        continue
```

```
    if length < MINIMUM_RECOMMENDED_LENGTH:
```

```
        # Soft advisory only, not a hard block - the brief's own
```

```
        # example uses 8 characters, so rejecting short lengths
```

```
        # outright would contradict its own spec. Modern guidance
```

```
        # (NIST SP 800-63-4, 2024) recommends 15+ for high-security
```

```
        # use, but the choice is left to the user here.
```

```
        print(f>Note: {length} characters is short for a real "
```

```
            f"password - {MINIMUM_RECOMMENDED_LENGTH}+ is safer.\n")
```

```
    return length
```

```
def generate_password(length):
```

```
    """
```

```
    PROCESS: build the password.
```

string.ascii_letters + string.digits gives a 62-character pool

(52 letters + 10 digits) without manually typing every character

out by hand.

`secrets.choice()` - not `random.choice()` - selects each character, since ``secrets`` draws from the operating system's secure entropy source instead of a predictable pseudo-random sequence.

Characters are collected with `.append()` (the Project 1 list skill) and assembled once at the end with `".join()",` instead of using `password += char` in a loop. Since strings are immutable in Python, `+=` would force the creation of a brand new string object on every single character - expensive as length grows. Building a list first and joining once avoids that entirely.

```
"""
```

```
character_pool = string.ascii_letters + string.digits
```

```
password_characters = []
```

```
for _ in range(length):
```

```
    password_characters.append(secrets.choice(character_pool))
```

```
return ".join(password_characters)
```

```
def main():
```

```
    print("\n" + "-" * 36)
```

```
    print("    RANDOM PASSWORD GENERATOR")
```

```
    print("-" * 36)
```

```
    length = get_password_length()
```

```
    password = generate_password(length)
```

```
    print(f"\nYour generated password ({length} characters):")
```

```
    print(password)
```

```
if __name__ == "__main__":  
    main()
```